Nama : Diky Andrianto

NIM : 95202532N

Growth Mindset

LK. 4.3 Project-Based

LATIHAN BERBASIS PROYEK dalam mempertajam POLA PIKIR BERTUMBUH

Tahap 1: Menentukan Pertanyaan Mendasar

Pertanyaan Mendasar (Essential Question): "Bagaimana pemikiran komputasional (*Computational Thinking*) membantu kita menguraikan masalah rumit menjadi langkah-langkah sederhana, sehingga kita berani mencoba dan tidak mudah menyerah saat menghadapi pesan *error* dalam proses *coding*?"

Tahap 2: Mendesain Perencanaan Proyek

Langkah 1: Topik Pelajaran dan Jenjang Kelas

- **Mata Pelajaran:** Informatika
- **Jenjang / Fase:** Kelas X / Fase E
- **Topik:** *Problem Solving* & Dasar Pemrograman (4 Pertemuan: CT ➔ Flowchart/Pseudocode ➔ Algoritma/Dasar Python ➔ Coding)

Langkah 2: Rancang Strategi Pembelajaran dengan 4 Komponen Utama

Komponen	Desain dalam Pembelajaran Anda DOCX
Cognitive Framing	Guru membuka kelas dengan *framing*: "Menulis kode di komputer adalah tahap paling akhir. *Programmer* hebat menghabiskan 80% waktunya untuk berpikir (CT) di atas kertas. Jika programmu *error*, itu

	bukan karena kamu tidak berbakat, melainkan logikanya perlu dirunut ulang dari awal."
Scaffolding	Siswa tidak akan menyentuh Python sebelum logikanya matang. Tahapan dijaga ketat: 1) Membedah masalah (CT), 2) Menggambar simbol (*Flowchart*), 3) Menulis rancangan bahasa manusia (*Pseudocode*), 4) Menerjemahkan ke sintaks Python (*Coding*).
Opportunity-Based Thinking	Pada tahap *Flowchart* dan *Pseudocode*, siswa saling menukar pekerjaannya untuk mencari "celah" atau cacat logika dari algoritma temannya sebelum dieksekusi di komputer. Menemukan kesalahan di atas kertas dirayakan sebagai "penyelamat" sebelum *coding*.
Formative Assessment	Penilaian difokuskan pada proses runutnya logika (kemampuan dekomposisi masalah dan alur *flowchart*), bukan sekadar pada kode yang 100% berjalan tanpa *error* di tahap akhir.

Langkah 3: Tambahkan Strategi Pendorong Growth Mindset

1. **"Paper Debugging" (Merayakan Revisi Kertas):** Siswa diajak untuk berani mencoret-coret dan merevisi *flowchart* mereka berulang kali. Kertas yang penuh coretan perbaikan dihargai lebih tinggi daripada yang langsung bersih, karena menunjukkan proses berpikir bertumbuh.
2. **"Rubber Duck Debugging" (Metode Bebek Karet):** Saat masuk ke tahap *coding* (Pertemuan 4), siswa wajib membacakan baris kodenya dengan suara pelan kepada teman sebangku sebelum mengeluh frustrasi dan memanggil guru.

Kasus Pembelajaran: Siswa dihadapkan pada masalah dunia nyata: "Merancang sistem mesin kasir sederhana untuk kantin sekolah yang bisa menghitung total diskon jika murid membeli lebih dari 3 barang." Seringkali, jika langsung *coding*, murid akan kebingungan menyusun rumusnya, *error*, dan menyerah.

Tabel Kegiatan Pembelajaran (Untuk 4 Pertemuan Berurutan)

Tahap / Pertemuan	Aktivitas Pembelajaran	Tujuan Pemilihan Aktivitas Belajar
Pertemuan 1: Computational Thinking (CT)	Siswa membedah masalah mesin kasir (Dekomposisi), mencari pola perhitungan diskon (Pengenalan Pola), dan membuang informasi tidak penting (Abstraksi). Semua dilakukan secara diskusi tanpa komputer.	Membangun keyakinan bahwa memecahkan masalah besar dimulai dari membaginya menjadi masalah kecil yang bisa diselesaikan satu per satu.
Pertemuan 2: Flowchart & Pseudocode	Siswa mengubah langkah CT dari pertemuan 1 menjadi simbol *Flowchart* dan *Pseudocode*. Guru bertanya: *"Apa yang terjadi jika kasir salah memasukkan harga di awal alur ini?"*	*Scaffolding* visual. Siswa menyadari bahwa memperbaiki kesalahan logika di diagram jauh lebih mudah daripada merevisi *coding* nanti.
Pertemuan 3: Algoritma & Dasar Python	Guru mengenalkan sintaks Python dasar (Variabel, Tipe Data, Operator). Siswa "menerjemahkan" *Pseudocode* mereka ke Python di atas kertas (*Dry Run*).	Menghubungkan logika manusia dengan bahasa komputer sebelum masuk ke tahap rentan stres (komputer).
Pertemuan 4: Coding & Debugging	Siswa mengetik di IDE Python. Saat muncul *error* merah, guru memandu: *"Lihat baris yang ditunjuk, cocokkan dengan flowchart-mu. Bagian mana yang terlewat?"*	Melatih resiliensi (*grit*). Menjadikan *error* sebagai *feedback* langsung dari komputer, bukan sebagai tanda kegagalan.

Penilaian (Asesmen)

- **Kognitif:** Rubrik kelogisan *Flowchart* dan keakuratan translasi ke *Pseudocode* / Python (Pertemuan 1-3).
- **Psikomotor/Proses:** Kemampuan melacak dan memperbaiki *error/debugging* mandiri pada Pertemuan 4.
- **Afektif:** Lembar observasi kegigihan siswa saat kode tidak berjalan sesuai harapan dan kemauan merevisi logika.

Tahap 3: Menyusun Jadwal

Minggu / Pertemuan	Aktivitas Utama	Output yang Diharapkan
Pertemuan 12	Menyusun pertanyaan mendasar dan merancang desain modul CT & *Flowchart*.	*Essential question* dan RPP Pertemuan 1 & 2.
Pertemuan 13	Mengisi LK 4.3 - Mendesain modul lanjutan (Dasar Python & *Coding*) berlandaskan *Growth Mindset*.	RPP Pertemuan 3 & 4 beserta strategi *Scaffolding*.
Pertemuan 14	Melakukan penyempurnaan LK 4.3, *peer-review* instrumen asesmen proses (*debugging*).	Finalisasi desain dan rubrik penilaian berfokus pada proses.
Pertemuan 15	Presentasi hasil proyek desain pembelajaran RPP 4 Pertemuan berurut dan refleksi.	Paparan desain kelas dan dokumen LK 4.3 yang utuh.

Tahap 4: Memonitor Kemajuan Proyek

(Jurnal Refleksi Mingguan) "Tantangan terbesar minggu ini adalah menahan diri untuk tidak langsung mengajarkan Python di pertemuan pertama. Mengubah kebiasaan siswa untuk bersabar membedah *Computational Thinking* dan *Flowchart* membutuhkan narasi yang kuat agar mereka tidak merasa bosan. Kemajuan minggu ini adalah berhasil merancang rubrik penilaian yang menghargai 'kertas coretan *flowchart*' sama berharganya dengan 'program yang selesai'. Teman sejawat menyarankan agar di Pertemuan 4 (*Coding*), batasan waktunya diperlonggar agar siswa tidak panik saat menemui *syntax error*."

Tahap 5: Menguji Hasil

(Fokus Paparan untuk Pertemuan 15) Paparan akan menampilkan alur yang logis:

1. **Mengapa CT Dulu?** (Menjawab Pertanyaan Mendasar).
2. **Strategi 4 Komponen Beruntun:** Membedah bagaimana *Scaffolding* diterapkan tidak dalam 1 hari, melainkan dipecah dalam 4 pertemuan (Dekomposisi CT ➔ Visualisasi Flowchart ➔ Translasi Python ➔ Eksekusi Coding).
3. **Strategi Tambahan:** Menampilkan bukti "Coretan Revisi Kertas" (*Paper Debugging*) sebagai budaya kelas.
4. **Evaluasi:** Menjelaskan bahwa siswa yang kode akhirnya *error* tetap bisa mendapat nilai tinggi jika CT dan *Flowchart*-nya terbukti logis dan sistematis.

Tahap 6: Mengevaluasi Pengalaman Belajar

Refleksi Akhir

"Tantangan terbesar selama merancang proyek ini adalah membongkar ekspektasi bahwa pelajaran pemrograman harus selalu di depan komputer. Merancang 4 pertemuan berurutan (dari CT hingga *Coding*) membuat saya menyadari bahwa frustrasi siswa selama ini terjadi karena ketiadaan jembatan berpikir (CT & *Flowchart*).

Setelah merancang pembelajaran berbasis pola pikir bertumbuh ini, saya merasa jauh lebih percaya diri. Saat mengajar nanti, saya akan secara radikal mengubah cara saya menilai. Saya tidak akan lagi terpaku pada apakah program siswa sukses beroperasi 100%, tetapi pada bagaimana mereka merespons pesan *error* dan melacak letak kesalahan tersebut kembali ke *flowchart* mereka. Pelajaran utamanya adalah: bahasa pemrograman bisa usang

dan berganti, tetapi kemampuan menstrukturkan pikiran (*Computational Thinking*) dengan gigih adalah keahlian yang akan terus bertumbuh dan relevan seumur hidup.